Chapter 13. Automation and Tooling

By Alex Chiang, Bharath Modhipalli, Daniel Hobe, Joanna Wijntjes, Michael Harris, Michelle Brush, Robin Roth, and Trevor Mattson-Hamilton
with Carmela Quinito

Automation is a key aspect of SRE. SREs should strive to engineer the systems they own rather than simply operate them. Many, if not all, operational tasks can be automated.

Reasons to automate include:

Saving time during normal operations

The main reason commonly given for automation is to save time. Productivity gurus can point out automatable aspects in practically any aspect of life and work. Any continuously running system comes with maintenance tasks. Daily or weekly tasks are very present to the engineers responsible for performing them, and it's part of SRE culture to despise repetitive work and consider how to "automate yourself out of your current job." While these types of tasks are likely the first that come to mind, the frequency or annoyance of a task should not be the sole motivating factor; consider also the three reasons that follow.

Saving time in emergencies

Automation can reduce the time it takes to detect, triage, mitigate, or resolve an incident. While the savings in terms of time might seem small, their business impact can be large. Ensuring that a full outage of a large online shop on Cyber Monday gets mitigated after 5 instead of 15 minutes will likely be more important to the business than cutting an hour from a weekly release qualification process. Some tight SLOs are defendable only through full automation. Automated systems can take action within seconds, whereas human operators typically need many minutes, depending on the complexity of the problem.

Consistency

Human operators can make mistakes or miss a step, particularly in high-stress situations. Automation improves consistency, making processes more predictable

and repeatable while also facilitating monitoring. This consistency is particularly important during incidents, as emergency mitigations often come with elevated privileges and thus increased risk. The stress of dealing with a never-before-seen outage might cause an on-caller to make mistakes, while automation (even if it handles only a subset of the mitigation steps) will stoically move along.

Scalability

Once automated, processes can be scaled, either horizontally (e.g., updating 1,000 servers in parallel) or by integrating them into higher-level automation. Eventually, a collection of tools, automation, and opinionated configuration can evolve into a platform, with a central team maintaining capabilities that many downstream teams can use. With a larger user base, the return on investment for automation improves. For example, if automated monitoring of exceptions raised in production systems gains a feature to generate code fixes for common issues, all users of the platform will benefit.

These reasons should all be weighed when considering what to automate. In "When to Automate and When Not To", we'll discuss how to decide on concrete projects.

Levels of Automation

Automation evolves and should always be driven by requirements. This evolution typically goes through the following levels:

No automation

A proof-of-concept application does not require automated database migrations, and it might even be fine to push updates manually. Similarly, for a service at the end of its lifecycle, it's likely not worth investing in automating service-specific processes.

Partial automation

As cases for automation are identified, they will be implemented progressively. Even partial automation can provide significant benefits, and it may be sufficient for infrequent procedures where fully automating the process would be difficult or infeasible. For example, per-customer customizations of a software product for its initial launch might best be partially automated. In this case, it's reasonable to retain some manual steps, especially if that means shipping to the first clients a day earlier.

There can be a risk of getting stuck at partial automation (see "Incremental Automation"), however, which can prevent the key benefits of consistency and scalability.

Full automation

Automating all steps lets you reap all the benefits and move on to either automating at a higher level or scaling the system or SRE engagement with these new capabilities. Full automation is the key to *sublinear scaling*, which here means increasing business impact faster than the human effort required.

Full automation can take different forms:

Local automation

As a team builds automation, it often starts by automating the operational tasks associated with the services it owns. Only that team benefits from this automation.

Platform automation

Moving automation to a shared platform owned by a central team or offered by an external provider removes the maintenance burden from local teams and allows the benefits of automation to scale.

Inherent system behavior

Some kinds of automation can be made redundant by making them inherent system behavior. For example, a job syncing data between two databases becomes unnecessary if the architecture is changed so that all components access a single source of truth.

If time savings are the only consideration, you can use the famous "Is It Worth the Time?" calculator from *xkcd.com/1205*, shown in Figure 13-1. In practice, however, SRE teams should also consider the consistency and scalability benefits discussed earlier.

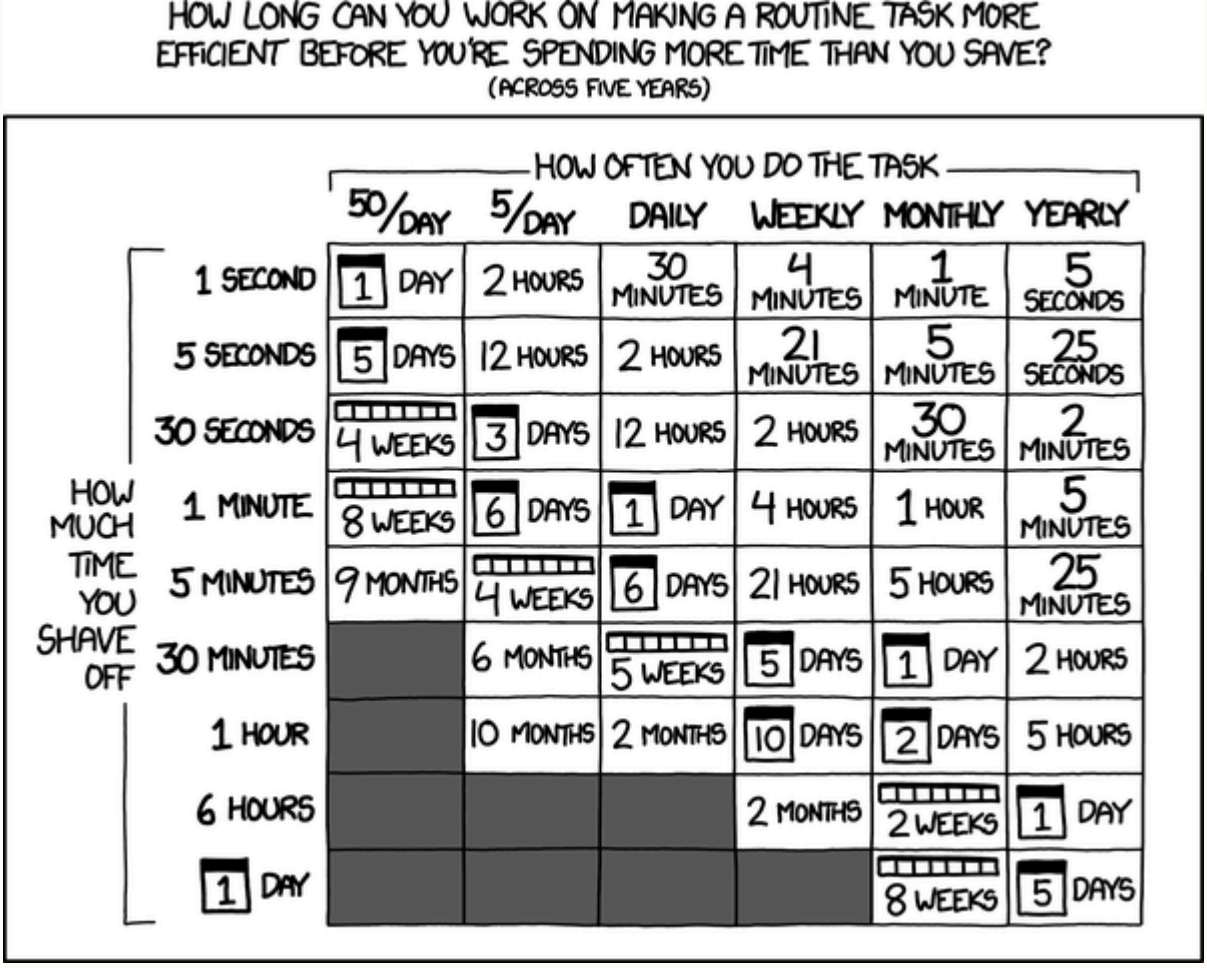


Figure 13-1. The xkcd "Is It Worth the Time?" calculator

Scaling Automation

Identifying an automatable flow often creates a strong impulse for engineers to streamline it, and they derive immediate satisfaction from implementing a quick, local solution. This can result in the development of numerous local tools that are far removed from the core business logic and known to only a small number of experts, with high maintenance costs. If an issue can be solved at the platform level at only a marginally higher cost than with local automation, directly implementing the long-term solution will yield a greater return on investment. (For more details on this, see "Incremental Automation".)

Maintenance costs of local automation can be reduced by generalizing and sharing solutions. Even if only two teams share an automation solution and alternate feature development, the added attention in review and testing, as well as more frequent use, can make the difference between a brittle tool and a robust one.

Larger organizations often establish central teams to develop shared automation and build platforms to make these capabilities available to everyone.

Example: Capacity Provisioning

SREs at Google are typically responsible for planning and provisioning the capacity of the services they own (see Chapter 15). This includes choosing locations and sizing strategies. The automation lifecycle often includes these four phases:

1. *Internal prototype*

   Services run in a single location with a hardcoded number of replicas.

2. *Beta*

   Add location redundancy and enable horizontal scaling with automatic sizing.

3. *GA*

   Run services in all relevant geographic locations, estimate inorganic growth at launch, preallocate capacity, and shift sizing from replica numbers to customer usage metrics.

4. *Mature*

   Continuously optimize and automate placement and sizing to ensure redundancy while minimizing cost.

As the requirements for capacity provisioning tooling are shared by many services, automation can benefit from being developed at the platform level.

Google uses two layers of independent automation tools for sizing services on its internal platform:

- At short timescales (minutes to hours), resource limits and replica counts are adjusted based on current usage (similar to horizontal and vertical Pod Autoscaling in Kubernetes).
- For longer timescales (days to months), demand forecasts are combined with expected inorganic demand from launches and matched with existing orderable supply to choose locations and propose sizing.

These automation tools are developed centrally and are part of Google's production platform. As such, service owners can get the benefits with minimal effort, and organizations can apply improvements at scale. Given these opportunities, the lifecycle boils down to:

1. Internal prototype

Build on the platform. Don't consider capacity planning; just use defaults.

2. Beta

The defaults are still good enough.

3. GA

Inorganic growth isn't modeled in the defaults, but once an estimate for users or requests exists, central tooling can translate it into resource amounts.

4. Mature

Small to medium services can run on the defaults and rely on fully automated capacity planning. Large services will often tune their configuration to explicitly balance cost against latency or availability, adjust the speed of resizes, and validate changes. They will also integrate closely with long-term resource planning and ordering.

In developing these tools, it's critical to choose the right abstraction layer. For automation to be generally useful, it needs to be abstract enough to reduce the complexity a human sees but still allow all necessary modifications. Most users won't need to tune the demand time-series forecast, but they might want to choose whether to plan for mean, 90%, or peak loads, depending on the service characteristics. Similarly, most users won't care which location is selected, but they might need to configure location flexibility and colocation requirements with dependencies.

The central automation of capacity planning ensures all of the benefits we discussed earlier:

Saving time during normal operations

Teams responsible for small and medium-sized services don't have to think about capacity planning. They might get notifications or requests to verify changes, but they don't have to decide which redundancy level to use or how to correctly forecast their demand. Even for large services, most relevant features exist and the team's focus can be on configuration rather than writing automation.

Saving time in emergencies

At scale, rare cases become common, so tools for emergency capacity changes are available and integrated with continuous capacity planning.

Consistency

Automated capacity planning does not overlook locations or misinterpret graphs. Forecasts about resource consumption rely on consistent and predictable automation. Simulations and A/B tests become reliable predictions when they are built on top of consistent platform-level automation instead of changes to manual processes.

Scalability

Capacity planning at the platform level makes this automation available to all services. At scale, more features are worth implementing, from supporting new resource types to modeling dependencies between compute and storage or automating moves between locations.

Example: Incident Management

On-callers need to respond to incidents with a focus on managing them through all stages of the incident lifecycle (Figure 13-2). Following this process ensures learnings are translated into system improvements intended to materially reduce the likelihood of the same type of incident recurring.

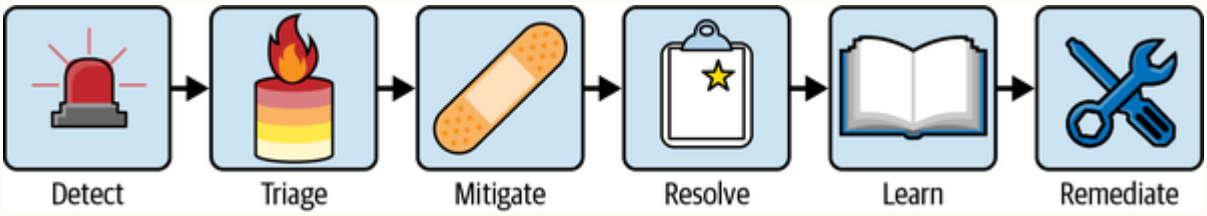


Figure 13-2. The incident learning feedback loop

Over the past decade, this experience has evolved for on-callers at Google: we've transitioned from using multiple focused tools to using a common Incident Response Management (IRM) platform.

Our IRM platform supports the entire incident lifecycle, making it easier to automate more on-caller journeys with greater awareness of the state of incidents across the company. On-callers benefit from the platform's guidance during stressful situations—they have to rely less on remembering training nuances, while automation helps simplify their actions. As a result, it's easier to follow Google's incident management processes (see Chapter 10). Although effective human response remains a crucial skill, automation assists an on-caller focus by increasing repeatability, expediting processes, ensuring consistency, and improving visibility for stakeholders and customers across the organization.

The following examples illustrate how platform support for extensibility enables automation integrated into incident management journeys:

Automating impact analysis and contextual insights

Alerts are enriched with correlating signals from all observability sources to flag customer impact, highlight anomalous patterns, and correlate related incidents in system dependencies based on the production knowledge graphs (see "Observability and Monitoring"). This data-driven awareness of the real-time situation, paired with relevant historical context, accelerates triage and reduces the time to effective mitigation.

Facilitating action

To enable effective response to incidents spanning multiple systems or requiring expertise from multiple teams, automation can auto-assign incident roles to assemble the core response team. Automation can also establish dedicated coordination artifacts (shared docs) and communication channels aligned with team and organizational best practices. Additionally, based on the nature of the alert, automation can suggest and trigger appropriate *automatic mitigation* actions (e.g., roll back, shift traffic), thus minimizing time to mitigation (see "From Investigation to Automatic Mitigation").

Streamlining communications

Automation supports status updates to leadership, support teams, and other stakeholders to ensure broad, consistent, and timely awareness of the incident's impact and progress with fewer interruptions for the response team. Customer impact assessment automation aids emerging incident detection to accelerate customer communications.

Enhancing learning

Automation captures key events—such as impact declarations, mitigation actions, and responder notes—during incidents to autopopulate the postmortem and guide the learning process. This aids postmortem authors in quickly moving beyond documenting what happened to analyzing and understanding the broader sociotechnical system (see "Evolving the Learning from Incident Practices"). That way, on-callers can more quickly identify contributing causes and create effective action items that convert lessons learned into concrete preventative measures and system improvements.

Each of these examples of targeted automation brings direct value to on-callers within their current focus without requiring additional context switching. Requirements and applications for your team may lead to automation showing up differently for you, just as it varies across teams at Google.

By automating aspects of incident response, IRM fundamentally alters our work and our workflows and reduces the time spent on rote analysis, comms, and process. As a result, we're able to focus on areas where human judgment, knowledge, and collaboration are key.

When to Automate and When Not To

The previous section covered why you should automate. To get a complete picture, however, you must also consider the pitfalls. While the SRE instinct is to automate everything, a disciplined approach is required to select the right tasks. Unchecked, automation can introduce new risks, mask underlying problems, and ultimately create more work than it saves. This section provides a framework for making deliberate, effective automation decisions.

THE IRONIES OF AUTOMATION

Lisanne Bainbridge's paper "Ironies of Automation"[1] is critical to understanding how automation impacts organizations and systems. This seminal research paper looked at the impact that automating industrial processes had on the workers who originally performed the work.

Bainbridge found that automating away work creates whole new classes of work that didn't exist before the automation—notably, monitoring it and repairing it when it breaks. This new work is often harder than the original task that was automated.

Additionally, when work is automated, lack of use can cause operators' skills to atrophy. This makes system troubleshooting and incident response even more difficult.

The ironies, therefore, are that automation can create new work, cause skill decay, and require organizations to train or hire people with higher-order skills to deal with failures and anomalies. This doesn't mean you shouldn't automate; it's just an additional consideration when deciding whether to automate.

The Pitfalls

As mentioned previously, most operational tasks can be automated, but it's important to be aware of the potential downsides. Before embarking on an automation project, consider these common traps:

Dangerous automation

Automating systems that control mission-critical functions without robust safety controls is a recipe for disaster. Bugs, flawed configurations, or unexpected data can trigger catastrophic failures that a human operator might have prevented. The more powerful the automation, the more sophisticated its safety mechanisms must be. These may include rate limiting, enforceable scope limits, "do no harm" checks, and circuit breakers that halt automation and escalate to a human when predefined safety thresholds are breached. For more on holistically analyzing a system to identify dangerous automation (among other issues), see Chapter 12.

Deskilling operators

Automation is almost never complete, and automation that is complex but incomplete creates a paradox. It may handle 99% of cases, meaning human operators rarely need to intervene. However, when that rare edge case occurs, operators may have lost the skills and situational awareness required to intervene effectively. This phenomenon, described in the sidebar "The Ironies of Automation", is a critical human factors consideration—the less a human does, the less they may be capable of doing when needed most.

Automation surprise

Automated systems do not always do what the operator expects them to do. Therefore, automation does not eliminate human work—rather, it shifts humans from an active role to a supervisory role. Without sufficient observability, automation can surprise the operator with its behavior and make resolving issues extremely difficult.

Automating over a broken foundation

Using automation as a bandage for a fundamentally broken system is a common trap. It can extend the lifespan of a flawed architecture by making it *seem* sustainable. While the automation may reduce operator toil, the system's underlying limitations will eventually block larger business objectives, and the investment in automation will be lost when the system is inevitably replaced.

Automating a moving target

Automation delivers value over time. Automating a system with a short lifespan will likely yield a poor return on investment, as will automating a system with low process stability (i.e., one whose implementation or surrounding processes change frequently). The initial engineering cost and the ongoing maintenance required to keep the automation in sync with system changes can easily overwhelm the time saved. Furthermore, this constant change makes the automation brittle, increasing the risk that it will fail unexpectedly—often at the worst possible time.

Automating ambiguity

In general, automation excels at deterministic tasks. It fails when it must interpret ambiguous signals or make decisions that require nuanced human judgment. As discussed in Chapter 21, AI is continually pushing the boundary of how much ambiguity automation can handle, but recognizing the limits of the current state of the art is critical. Attempting to automate tasks beyond those limits will result in mistakes, with the toil required to correct them outweighing what was saved by the automation itself. This pitfall is deeply intertwined with the "ironies of automation" discussed in the earlier sidebar, as intervening in a misbehaving automated system is often more complex than performing the original manual task.

A Framework for Decision-Making

Deciding when to automate requires a balanced approach that goes beyond simple time-saving calculations (see Figure 13-1). The following framework for making these decisions combines a quantitative analysis with a qualitative evaluation.

Quantifiables

The foundation of any automation decision is a cost-benefit analysis. While the inputs often require estimation, this formula provides a clear starting point:

$$Costs = cost_{initial} + cost_{ongoing} \times system\ lifespan$$

$$Savings = time_{task} \times frequency \times system\ lifespan$$

While savings exceeding costs is a great initial indicator for the viability of a project, engineers tend to underestimate both initial and ongoing costs—especially the effort needed to adapt automation to evolving systems—so you must take care when performing this estimation. Furthermore, a positive return on investment is not a green light; it is simply an input into the final decision-making process. Not only must you evaluate the

nonquantifiables, as described in the next section, but you also must always weigh an automation project against other potential automation projects to ensure the team is tackling the highest-impact work first.

Nonquantifiables

A purely quantitative analysis is rarely sufficient. Sound judgment is required to weigh factors that are difficult to model financially, such as:

Strategic value (speed and consistency)

Sometimes, automation that shows a net-negative return on paper is strategically essential. Consider applying security patches to a large fleet. The financial risk of a data breach from a slow or inconsistent rollout is immense, though difficult to quantify precisely. In these cases, speed and consistency are features in themselves, and automation is the only viable way to achieve them at scale.

Hidden costs (risk mitigation)

When considering automation, you must also price in the cost of mitigating its risks. A powerful system needs robust safety system mechanisms. The cost of designing, building, and maintaining these safeguards is a critical—and often overlooked—part of the total investment.

The ambiguity factor

If the automation relies on ambiguous signals, the quantitative model is likely flawed. You will overestimate the frequency with which the automation succeeds (and thus saves time) and underestimate the ongoing maintenance costs of handling its mistakes.[2]

The annoyance factor

A purely quantitative analysis might miss the human cost of certain types of toil (for more on toil, see "Toil" and "When to Toil and When Not To"). A task that takes only five minutes but frequently wakes an on-caller in the middle of the night or causes continual context switching during the day has a disproportionately negative impact on morale, focus, and team health. Automating such tasks can provide significant relief, even if the total time saved seems small on paper.

For more on evaluating the benefits of reliability and the factors that contribute to it, see Chapter 2.

Automation Examples

Combining the quantifiable and nonquantifiable elements requires experience and judgment. The following examples illustrate this process in practice.

Automated hardware fault mitigation

Automating network hardware fault mitigation demonstrates a blend of quantifiable and nonquantifiable analysis. In theory, human operators could react to alerts and apply configuration changes to route traffic around faulty components on the fly. On a small network, this might be feasible.

The quantifiable savings of automation are clear (reduced operator time, avoidance of SLA credits). However, the dominant factors are nonquantifiable. First is speed. Every moment a fault goes unmitigated, customers may experience degraded performance or a complete outage, leading to lost revenue (partially quantifiable) and reputational damage (nonquantifiable). For Google's services, the business case was overwhelmingly positive.

However, automation introduces its own significant risk: a faulty signal (e.g., a detection bug alerting on all hardware in the fleet) could cause the automation to route traffic away from healthy infrastructure. At scale, the automation could trigger an outage far larger than the one it was designed to prevent. Consequently, in our case a core part of the project was a sophisticated safety system that stops the automation and alerts a human if its actions exceed predefined safety thresholds (e.g., "do not drain more than X% of capacity in a region"). Over time, the number of overlapping checks has grown, making it difficult to reason about what operations will be permitted, and the system often unnecessarily prevents capacity from being repaired. Maintaining and debugging the checks represents a huge, ongoing cost that likely would have been overlooked in a superficial analysis.

Automated regional rollout system

Over the past decade, Google has pushed to segregate systems into independently deployable regional copies to limit the blast radius of software and configuration rollouts. Initially, the available deployment automation was highly flexible but required teams to cobble together custom scripts to achieve regionalization. The significant up-front and ongoing maintenance costs meant that only teams managing the highest-risk services could justify the investment. For hundreds of other services, the cost of regionalization was simply too high.

At the same time Google was pushing for much higher levels of reliability, a new, more opinionated framework for production deployments, which grew out of a grassroots effort, was gaining traction. This framework was called *Production on Demand* (Pod). The convergence of identifying the need for greater levels of automation and consistency across services and the availability of Pod led to transformational changes in how services are deployed at Google.

Standardizing deployment automation on a single platform enables that single platform to universally provide and maintain drop-in regionalization capabilities. Although this standardization push was costly for existing services, it was pursued as a strategic initiative. The goal of standardization wasn't just to reduce toil; it was to *change the cost-benefit analysis for every service at Google*. By dramatically lowering the cost of regionalization, standardizing on Pod made it feasible for hundreds of low- and medium-risk services to adopt a more reliable posture. The aggregate reduction in reliability risk across the entire company was substantial, providing a clear strategic justification that transcended any single team's budget.

Automating restarts for a memory leak

A common antipattern involves using automation to mask underlying systemic issues rather than fixing them. Consider a service with a known memory leak. As memory usage climbs, performance degrades until the process crashes. You can implement automation that proactively restarts the service when its memory usage crosses a certain threshold as a solution.

This approach must be evaluated carefully. If the memory leak is rare and the cost to fix it is high (e.g., requiring months of engineering time), this automation might be a pragmatic, cost-effective solution. However, it becomes a dangerous antipattern when it masks frequent or escalating problems. If the leak requires restarts significantly more frequently than the normal rollout cadence—for example, if the restarts move from daily to hourly—the automation is simply creating technical debt and hiding the urgency of fixing the fundamental problem. The strategic approach is always to fix the underlying causes, but the tactical decision depends on the cost-benefit analysis.

Incremental Automation

Automation is sometimes seen as the ultimate solution for improving efficiency and reducing toil. However, we caution against attempting to automate everything all at once. A more effective strategy involves a phased, incremental approach that not only stream-

lines processes but also builds trust, improves design, and fosters a culture of continuous improvement. This approach can be summarized by the philosophy of "see one, do one, automate one," where a manual process is observed, executed, and then gradually transformed into a fully automated solution.

From Toil to Automation: A Practical Framework

A foundational step in this journey is the establishment of a structured review process. A regular production meeting (see the sidebar "Production Meetings"), where teams analyze recurring issues like bugs, pages, and other forms of toil, can provide a crucial feedback loop. By reviewing the pages and interrupts from the previous period (e.g., week, month, or quarter), teams can systematically identify the manual, repetitive tasks and pinpoint prime candidates for automation. This practice ensures that automation efforts are focused on the most significant pain points, rather than on trivial tasks. It's also an opportunity to do some data collection that ensures you are accurately identifying the most impactful opportunities for improvement from among those tasks. Engineering availability is limited, so careful prioritization (as described in the next section) is vital.

Once a candidate task is prioritized for improvement, the automation process begins. The first step is to perform the task manually. This initial phase can be an important learning experience, helping engineers gain a deep understanding of its nuances, edge cases, and failure modes. This "see one, do one" stage lays the groundwork for a robust solution. Hands-on knowledge then translates into scripting, where the manual steps are codified into a series of commands. Initially, these scripts might be rough and require manual review or a person to trigger the final action. Over time, they evolve into more sophisticated, integrated solutions, culminating in a fully automated process.

There can be some resistance to testing some of these scenarios. For instance, if the goal is to drain a region in the event of a certain failure, people may not want to actually drain the region as a test of the automation—they rightfully worry about creating an outage as they work to address the automated mitigation for an outage. Another version of this apprehension shows up during DiRT testing (discussed in more depth in Chapter 33 of the SRE Book) when people choose modest, safe targets for their disaster testing because they don't want to create an actual disaster. This route exchanges a short-term maybe-win for a long-term loss. The automation, and the people involved, need to be able to handle the mitigation (say, draining a region) confidently and consistently, and it's important to gain this experience outside of a crisis. Do the necessary work to ensure that the test is as safe as possible, but don't avoid running it because you're worried about the outcome.

Prioritization: The Automation Snowball

When it comes to prioritizing a set of automation projects, a common approach is to look for the highest ROI, or the biggest bottleneck. While logical, this prioritization often leads to tackling the largest, most complex projects first. If a team is already constrained by operational load, it may struggle to find the time to complete a large project, delaying any savings.

An alternative strategy, often more effective at getting buy-in for automation work, is to gain traction by instead taking a fastest-first approach. Start by tackling quick wins to demonstrate value. Prioritize the automation projects that can be completed the fastest, even if their individual ROI is smaller. This approach delivers immediate value and, more importantly, buys back engineering time. This recovered time can be reinvested into the next automation project, creating an "automation snowball"—a positive feedback loop where capacity gains accelerate over time.

The Benefits of an Incremental Approach

An incremental approach to automation offers numerous benefits. It delivers value sooner by tackling smaller, manageable pieces of the problem first. This reduces the risk of a large-scale project failing to meet its goals, as each small success builds momentum. It also reduces the risk of skew, ensuring that the final automated solution remains closely aligned with the actual problem. The iterative cycle of manual action, scripting, and refinement allows the team to test assumptions at each stage and adjust as needed, leading to a better design that accounts for real-world complexities.

Perhaps most importantly, gradually introducing automation builds trust and adoption. People are naturally wary of new systems, especially those that take over critical tasks. By gradually introducing automation and proving its reliability at every step, teams can win over skeptics and build confidence. The ultimate goal is to have the automation be so reliable that it becomes the default, trusted method for the task.

At Google, the benefit of an incremental approach was seen during a significant process migration from a version of software running on every switch in every datacenter to a new control system. The migration was slightly different from one region to the next and involved numerous steps, many of which were both repetitive and finicky—a good opportunity for automation because software would be more accurate than a human. During the early stages of the rollout, the automation would at times open a ticket for the on-caller with instructions on what to check and an action to complete. Marking that task as done restarted the automation. We iteratively added automation to cover many

of these manual tasks, and by the time we entered the high-volume phase of the migration, very little manual work remained.

In addition to having many opportunities to observe the benefits of incremental automation, we've also had several opportunities to see the risks introduced by automation that impacted many systems in many places at the same time, resulting in serious outages. A number of contributing factors made these outages possible, but one lesson stands out: make sure your defaults do not act on everything.

We've encountered situations where a system was expecting several variables but received only a subset, or none at all. You might assume that your system will return an error, fail safely, or perhaps retry after determining that it hasn't received all the expected inputs. Instead, in these outages, the system acted on the variables it had, ignoring the missing ones and producing a surprising result. Rather than taking no action, the default behavior was to act on everything, everywhere. So, as you build automation, or possibly direct your favorite AI agent to build it, ensure that the default behavior is a safe and appropriate failure mode. In particular, verify how the system behaves when an important variable, such as the region, is left out.

Important Early Considerations

The earliest versions of your new automation will require additional safety mechanisms, monitoring, and supervision for best results. For instance, suppose you have a service that reacts to an alert and takes a certain action. A safe first step can consist of correctly and consistently identifying the scenario that you want to automate. Each time a situation arises where the automation would take action, it sends an alert to the engineering team members, who review it and check for false positives or false negatives. An incremental next step is to correctly identify the problem you want the automation to respond to and then suggest (but not do) the action, again notifying the engineering team, who can see if the problem was accurately discovered and if the suggested action was correct. Next, the automation can identify the problem, take the action it determines to be appropriate, and notify the engineering team so they can review the result. Operating in this manner has an additional benefit: even if only one or two engineers are building the new feature, everyone on the team will see these alerts and be able to follow along with the work in progress.

The Final Stretch and Fostering Collaboration

A common pitfall is the tendency to stop when a task is 90% automated. While it may seem like a great achievement, a partially automated process can be more problematic

than a fully manual one. It creates a state of confusion, where team members are unsure whether to trust the automation or revert to the old, manual way. The last 10% of an automation project is often the most challenging, because it's usually the gnarly bit no one wanted to deal with, or the corner case that's not well understood—but it's also often where the most value is unlocked, transforming a good solution into a great one that truly saves time and reduces errors.

Furthermore, partially automated systems can increase cognitive load. Operators must understand not only the underlying system but also the automation's behavior, its limitations, and when it requires intervention. This situation can lead to the scenario discussed in the sidebar "The Ironies of Automation", where the automation is complex enough to cause operator deskilling but not reliable enough to be trusted. You can mitigate the effect of this extra cognitive load by creating readable logs and visible dashboards with status pages. For instance, if your automation has successfully identified hardware that was blackholing traffic, and that part of the fabric has been drained, ensure that dashboards showing traffic for that fabric show the new status, ideally with a linkable event where the engineer can learn more.

Finally, effective automation isn't just a technical challenge; it can be a social one. If you are going to need contributions from neighboring teams, try to involve them in the goals of the project. They may not have cycles to contribute, but they can provide recommendations based on their own areas of expertise.

It's been our experience that great automation finds users and contributors. At Google, for example, many services had a requirement for automated testing of their database backup, and a variety of solutions were implemented, with different degrees of usefulness. Through a cross-team effort, from this mix of solutions, an efficient service for testing that backups were valid and mountable was written. As people discovered this service, they adopted it because it did a great job and reduced toil for their teams. There's never been a mandate to adopt it; it just gets used because it works well.

Coevolution of Distributed Systems and Automation at Scale

Traffic Director is a relatively modern[3] network infrastructure service that provides a fully managed and highly available control plane for Google Cloud's load balancing products. The story of its operational evolution is a lesson in how architectural choices can result in surprising interactions with underlying automation platforms that are supposed to just work.

Like many software projects, Traffic Director started life as a monolithic binary. The Traffic Director SRE team ran two separate instances of it, each to support a different tenant, which immediately required them to run separate copies of the rollout system, monitoring stacks, and probing. And like many software projects, the team had to scale it quickly, so they split its functionality into three separate binaries.

Over time, other product teams at Google grew interested in adopting Traffic Director, and the Traffic Director SRE team was faced with the problem of replicating everything by an order of magnitude. The Traffic Director product development and SRE teams debated about how to balance the product demands against the operational complexity, with the heart of the argument revolving around whether they should adopt a multi-tenant architecture or a multi-single-tenant architecture.[4]

For a young service like Traffic Director, supporting multitenancy for tenants with different scaling and reliability requirements would have required the team to spend significant effort focusing on resilience and least-common-denominator features rather than maximizing product velocity—a critical consideration when building for Cloud customers. Multi-single-tenant seemed like a better fit for the business requirements but was anticipated to be operationally daunting.

Enter Pod. Pod, or Production on Demand, is Google's internal production platform, which promises to deliver seamless integration with the internal Google ecosystem and provide improved automation, reliability, and operational efficiency through standard APIs and developer experience.[5] An engineer familiar with running one service on Pod should be able to easily run another service on Pod.

Pod offered a potential solution for Traffic Director: a single source of truth for configuration that could spawn all of its multi-single-tenant binary deployments. This solution is the SRE holy grail: sublinear complexity for superlinear growth.

And it worked! The Traffic Director SRE team consolidated *N* rollout systems, monitoring configs, and capacity configs down to one. Life was good for a few years. Until it wasn't.

Traffic Director continued to scale across multiple dimensions, including number of tenants supported, global and regional variants, and simply running in every new datacenter that Google built. All of these dimensions (and more) produced a massive cross-product in our single configuration, which made it the single largest configuration at Google.[6]

Simultaneously, the configuration itself was becoming unmanageable. Tenant-specific flags, all living in the same config and generated with snippets of code, immediately diverged. Sharding might have been enabled for one tenant, while caching was enabled for

another. The inconsistency wasn't just a matter of untidiness; it became a primary source of reliability risk, creating a combinatorial explosion of behaviors that was impossible to reason about.

This debt came due in the form of cascading platform failures. Core automation, the very promise of Pod, began to break because it couldn't handle our scale. The rollouts got so supremely wedged that we ran a two-month-old binary in production. The resulting postmortem had a section grimly titled "Blood and Toil" that described how engineers had to perform hundreds of manual operations to roll out a single weekly release.

Eventually, the Pod team intervened with a "white glove" program, and the Traffic Director SRE team partnered with them for over 18 months in an expensive, resource-intensive effort to stabilize the service. The Pod team fixed scaling issues in multiple places throughout the platform.[7] The Traffic Director SRE team adopted advanced features of Pod, including breaking up the large rollouts into smaller ones, trading complexity for reliability, implementing special settings to reduce the size of the configuration cross-product, and in some cases turning off parts of the automation altogether.[8]

While these changes improved Traffic Director and Pod stability, they were bandages over underlying issues. The Traffic Director SRE team eventually concluded that Traffic Director needed to evolve to a multitenant architecture, given the scale and scope it now had and the improvements in operational sustainability this would bring. The team also recognized that it needed to continue evolving the automation platform itself, as its systems represent a set of ever-changing requirements.

So what is the lesson here? Adopting a multi-single-tenant architecture was the correct choice at the time. It allowed the Traffic Director SRE team to maximize product velocity by decoupling tenant-specific features and improve reliability by isolating tenant blast radius. That architecture sufficed for over five years, which is several lifetimes in Cloud! But when the effort of maintaining the old architecture continued to increase, the team was able to realize it was time for a new design. It's easy to think of revisiting architectural choices as negative, but as this example shows, framing these decisions around SRE principles of automation and toil is a powerful technique that enables you to pursue the right architecture at the right time and identify when it's time for a change, ensuring efficiency and scalability throughout the lifetime of a product.

Toil

Addressing toil is vital to sustaining a productive and happy team. Understanding what you are dealing with, and addressing it proactively, will prevent outages, accelerate development time, and ensure engineers' workloads are more sustainable, even in complex or high-pressure environments. Failing to address toil creates a rapidly accelerating spiral of depreciating return that can quickly lead to poor velocity, increasing strain on the team and, in the worst-case scenario, causing it to implode.

So what is toil? Toil is manual, repetitive work that scales linearly with your service's growth and provides limited return on investment. Engineers think of it as work that isn't an important part of their goals for the quarter and that has no enduring value for the team or product. Common examples of toil include repeatedly rerunning a CI/CD pipeline because of a known flaky test, having to manually verify and restart a process that frequently crashes, or manually testing and validating a database backup process. A common entry point for toil is that it initially seems like a one-off: you have a new service, and to service the service from a new region you log in, allocate some memory or other resource, update, and log out. The entire process took just a few minutes and doesn't need to happen again until you perform setup in another new region. But the number of regions grows, the service grows, the requirements of the service expand, and it becomes a source of toil that is not providing value to the team or the service itself.

It's also important to talk about what toil is not: it is not a catch-all term for work a person doesn't like doing. Having to set up a new company-sanctioned security policy with reduced permissions may be finicky and tricky—but it's important for security, will show up on goals for the service, and has long-term value. Proposing and developing a bug fix for a recent outage may be challenging, but that doesn't make it toil. Adding integrated tests after an outage to prevent a problem from happening again may be difficult, but it isn't toil. This is engineering work—it requires human judgment, provides enduring value, and is an investment in the future reliability of the service.

Why Toil Is a Problem

Toil wastes your time. You can lose an entire day fixing an uncooperative release, getting flaky tests to pass, or ensuring that new features are deployed safely. Toil consumes valuable engineering hours that could be spent on strategic, innovative work. Worse, engineers stuck fixing trivial problems may lose enthusiasm for work generally, or stop looking beyond the daily weeds to see the bigger-picture solutions they should be driving toward. This scenario leads directly to burnout and attrition, which further affects

the team's capacity, creating a vicious cycle. Toil hinders the growth of the service and the company—a team bogged down by toil can't evolve its services or build new features.

Measuring Toil and Defining Success

To effectively address toil,[9] you need to quantify it, which unfortunately takes time. Ensuring that time gets allocated for gathering relevant metrics is vital, as the results will help you get to the contributing factors of your toil problems. You have to quantify the problem to know if your tactics are working, and in some cases to decide whether it's worth addressing at all. Numbers help you determine if you need to rewrite a tool or migrate to a new platform, or if the problem is not worth your attention.

Measuring Operational Workload

Start by measuring how much time your team spends on toil—track the time spent on specific tasks or survey the team. This baseline measurement is crucial because it lets you chart improvements over time and prove that your efforts are paying off. However you plan, account for the time when the SRE team is working on toil as well as new features, and track that number over time.

Measure unactionable alerts. If you have alerts that are not often actionable, it's probably because one in 20 identifies a critical outage. Everyone decides that catching that one outage is worth tolerating the other 19 alerts, but realistically, the alert needs to be improved so that the person receiving it has high confidence that it requires action. Review alert metrics regularly to determine what percentage were actionable and what percentage were noise, then track and improve that ratio over time.

Some of the most critical services at Google have a very low pager load, following years of work and improvements to monitoring and automation. When these services do page the on-call, it is because there is a noteworthy problem that requires addressing. This means no time is lost to unnecessary investigation and ensures decisive engagement from on-callers, who can have high confidence that a page means a problem and every second counts.

What Success Looks Like

The goal isn't to eliminate toil entirely, but rather, like with an SLO, to establish a manageable target and stay within it (see "On-Call Considerations"). Success means reduc-

ing the amount of time spent on toil to a sustainable level that frees up engineers to work on more valuable projects.

A foundational SRE principle is that teams should spend the majority of their time (more than 50%) on engineering work. The specific target should be tailored to the team's context, but the trend should be maximizing engineering time. For example, a good goal might be to reduce toil from 50% of an engineer's time to 30%. This shift significantly increases the capacity for high-impact work.

Beyond the numbers, success feels like relief. It's the transition from dreading an on-call shift to viewing it as a manageable responsibility. It's the feeling of empowerment that comes from eliminating a recurring problem rather than just mitigating its symptoms. A successful toil reduction effort improves team morale, reduces burnout, and creates space for innovation.

Effective Strategies for Addressing Toil

Successfully reducing toil requires more than just automating immediate pain points; it demands a shift in how the team operates. The strategies described in this section enable a framework for moving from reactive to proactive toil reduction. This involves not only addressing technical debt but also fixing inefficient processes and aligning team incentives to prevent toil from taking root.

Identify issues early

Whenever a new service is being developed, onboarded, or launched, work to identify and eliminate potential toil *before* it becomes a problem. One way is to establish a checklist so that the teams have a shared picture of expectations. Good questions to ask are: What does the monitoring and alerting look like for the service? How many pageable events were there in the last month? From there, you can map a path to actions: "Before launching this service, we should automate turning up the service in new locations," or "The service should be deployed on this common framework to make supporting it similar to other services we are on-call for."

Once the service launches, pay careful attention to the pages and tickets—both the ones you get (Do they need to be tuned? Too sensitive? Not directly relevant?) and the ones you did not get but should have gotten (e.g., if your entire home page was gone, but you were only alerted that your rollout failed). By building automation and robust systems from the start, you can avoid a lot of future toil.

Reward improvements

While fixing a single ticket is helpful, the most impactful work is fixing an entire class of problems. Reward engineers for addressing the contributing factors of the recurring issues, not just for handling the individual incidents. For example, if a certain type of error causes frequent pages, it's strategic to figure out how the problem arrived, look at where it was documented (or not documented), examine the code review process where appropriate, and proceed from there. Try to reward implementing solutions that are supported by a thorough understanding of what happens, as opposed to just trying to fix each instance as it occurs.

At Google, this type of work is often rewarded by nominating the SRE for a peer bonus—a small monetary award given by any Googler to another colleague to recognize outstanding contributions—but more strategically, it is also recognized by our performance and promotion criteria and planning process. Work to fix an entire class of problems would be scoped as a project, with success being met with all the same rewards delivering a new feature would be.

Recognize the real problem

Sometimes the toil isn't a technical issue but a procedural one. If your team has a policy of writing a detailed document after every incident, but those documents aren't leading to actionable changes, the toil might be the document itself.

Another antipattern is a team acting as a relay—just reading notifications and forwarding problems to another team. A key principle for Google SREs is that pages or interrupts should be actionable by the on-caller. If the action is to page another team, that team should take the page directly.

Analyze your processes with the same rigor with which you analyze your systems. If a process is not delivering value commensurate with its cost, eliminate or streamline it.

Set realistic long-term goals

Some toil problems are too big to be solved quickly. A major architectural change or a new release framework can't be built in an engineer's spare time. For these issues, you need to establish a suitable long-term goal and dedicate the necessary resources to it. Acknowledge that a major solution requires a focused effort and schedule it as project work, with defined goals and targets.

What doesn't work: Lessons and cautionary tales

The fact that we have written about this topic before and are writing about it again a decade later shows that the issue of toil by engineering teams is not a solved problem. Over time, we have found the previously mentioned suggestions to be effective. We've also tried the following strategies that have not worked well but seem to be easy traps to fall into:

"All hands on deck, it's an emergency!"

Declaring toil an emergency doesn't increase motivation—at least, not sustainably. In fact, it can add another layer of bureaucracy to an already frustrating situation, especially if it piles on urgency without headcount being allocated to actually address the problem. An emergency is an outage where a group of teams work together intensively to solve the problem quickly. Toil is almost never solved on that kind of timescale. Declaring an emergency for something that is going to stretch out for weeks, months, or more is a recipe for failure.

"Let's have everyone add data to some spreadsheets."

Creating a spreadsheet for every bad page or bug creates more work. Almost immediately, some people will start skipping the fill-out-the-spreadsheet step, and other people will probably step up and help. The resulting data will be too spotty to be useful, and the additional workload will be inequitable and distracting to what are likely some of your most diligent engineers. You've taken your toil and added a layer of toil frosting. Not good.

"If I pretend it's not there, it will go away" (also known as ignoring the problem).

Marking all toil as "obsolete" and hoping it won't return is just wishful thinking. Declaring bug bankruptcy is like deleting everything from your inbox without adding a good spam filter. The underlying issues will eventually resurface, and you still don't have a plan for fixing any of them.

Designating a "toil expert."

Chapter 30 in the SRE book describes a special case where an embedded engineer with sufficient expertise can step in for a measured period of time to help a team that's in trouble recover, explaining that this strategy can be effective because the team benefits from the outside perspective and decisive help. A common misapplication of this approach that we've observed to be problematic is to designate a single person on the team as the "toil expert," sometimes even routing all toil-type bugs to their queue because that person "is so good at it." This setup creates

a single point of failure, leads to burnout for that individual, and prevents the rest of the team from understanding the underlying problems. If they're shielded from the operational pain, they lose the context required to design better systems and prioritize toil reduction. Toil is a team problem, not a personal one.

"*Someone else will take care of it.*"

Outsourcing toil—creating a team whose job is to take care of the toilsome work, freeing you up to get back to project work—is a tempting strategy, but it often just postpones an eventual reckoning that may take the shape of a serious outage. If your toil is critical to your service's functioning and you outsource the work to a team that is not on-call for the service, you'll get paged for problems you now have no control over, and that team will be doing the work with little visibility into the expected outcomes.

"*But toil is a great way to learn.*"

This strategy is also known as "make it the new person's problem." New people joining your team deserve a healthy starter project; giving them your worst toil is more like hazing than onboarding and not a solid foundation for a healthy SRE team. While it's true that new members can gain valuable insights by tackling some operational tasks, there's a critical balance to be struck. Overloading newcomers with excessive or undocumented toil can be counterproductive, leading to burnout and preventing them from learning principles that are crucial for their long-term effectiveness. The goal should be to use toil as a teaching tool, not to treat new hires as a dumping ground for undesirable work.

When to Toil and When Not To

SREs do toil. In fact, toil can be necessary in the short term to bring the system to a better state as fast as possible. Engineering projects and incident response both have operationally shaped work that is similar in execution to toil. What's important is that the toil an SRE team takes on should be temporary and part of a broader plan to solve the problem that's causing it or automate it away. Toil that is routine and has no end in sight should be eliminated.

When to Toil

Sometimes toil can refer to undesirable work, but whether something is toil depends more on the enduring value of the work and how repetitive it is. If it's work that happens

rarely, requires human judgment, and has a high return on investment, then it's not really toil. For example, architectural migrations often require some level of operational work that feels like toil, but it's really engineering work. We need to distinguish between work that is necessary but manual, and actual toil.

That being said, sometimes repetitive work with limited long-term value is required as part of being an SRE. Incident response can involve toil. Ultimately, SREs are production experts who make use of their full toolbox of operational expertise and engineering acumen to solve problems. Here are some cases where toiling is necessary.

Toiling to mitigate

The most common example where toil is acceptable is when it's for incident mitigation. As discussed in Chapter 10, the goal of incident management is to understand the problem well enough to identify a generic mitigation that can be applied to get the system working again. For some situations, the generic mitigation takes the form of repeated manual interventions, and they can feel like toil. When this happens, SREs have three choices: fix the underlying system issues to resolve the incident, automate the generic mitigation so that it's applied without human initiation, or continue to toil until one of the other options can be implemented. Toil in this situation is a viable answer for a handful of days, but if used longer, it becomes a time sink and increases the risk of human error.

Google responders used toil for mitigation during an incident where they found that an issue was caused by a deadlock in one of the services. They suspected the deadlock was due to a recent change, but they needed more time to investigate. In parallel to the investigation efforts, the incident commander asked one of the SREs to monitor and manually restart the service's jobs when it looked like they were deadlocked. Eventually, the team identified the problematic deployment and rolled it back. The initial toil was needed to mitigate the situation and buy time to investigate.

Toiling to learn

Many people learn by doing. As such, performing operations manually for a short time is a great way for an SRE to better understand how the system works or to gain confidence in their ability to respond when automation fails. For SREs building automation, going through the process manually at least once helps them understand it. That said, when using toil to learn, people can easily fall into the trap of thinking that the path to automation is to automate exactly the steps they've performed manually. The goal is to de-

sign a system that eliminates the need for toil, not just to replace the manual steps with a script or command line tool.

An SRE team at Google used toil for learning when it wanted to automate its database recovery and data reconciliation efforts. Before writing any code, the team went through a full end-to-end recovery exercise, tracking every command required to complete the recovery. However, rather than just automating the commands as executed, the team used the logged commands to identify sets of logical operations that would be useful as generic mitigations for other types of incidents: blocking reads and writes, restoring the database, reconciling state with downstream data stores, and reenabling reads and writes. These logical operations were then designed as standalone automations that could be run independently or combined to perform the full recovery process.

Toiling to apply human judgment

Some operational decisions are so critical that they require a "human in the loop" to verify them before proceeding. Requiring a human to push a button or review a step might be necessary to prevent rogue automation from causing catastrophic damage.

In the database recovery and data reconciliation effort mentioned in the previous section, the team ultimately decided not to fully automate the recovery process because they wanted a person to make key decisions like when to initiate recovery. This decision turned out to be the right one, as shortly after the automation was created, a monitoring incident triggered every alert in the system. Had the team fully automated the recovery process, the monitoring glitch would have triggered the recovery automation, causing an unnecessary restore and resulting in recent user updates to the database being lost.

When Not to Toil

Although sometimes toiling is necessary or useful, SRE teams can find themselves doing it for the wrong reasons. These situations often look like they are temporary or part of a bigger plan, meeting the previously mentioned criteria of when to toil. However, in practice, they lead to the team absorbing more and more toil, without a clear end in sight. This section describes some warning signs to watch out for.

Toiling to close product gaps

The tools SREs create to mitigate incidents and manage systems are powerful. It can be tempting to use those tools to work around gaps in product design by creating whole features (or fixing poorly designed ones) through operational work. However, SREs

should not perform heroics to compensate for product gaps that make human toil a load-bearing and unscalable part of product behavior. Whether it's a missing feature or a design flaw, the engineering team needs to work quickly to close the gap rather than tolerating toil.

To illustrate toiling to close product gaps, consider a team defining a reservation feature. The team assumed customers would have no reason to cancel or change a reservation once the reservation period had started, so it didn't implement a way for customers to modify or delete reservations already in progress. However, within weeks of the feature's release, the team received numerous requests from customers to do exactly that. To fill the product gap, engineers had to manually modify the reservation metadata. This type of toil should be avoided as much as possible, and where needed, it should be conducted by the team that owns the feature to maintain the incentive to close the gap.

Toiling that scales with the system

SRE teams should always be wary of toil that grows faster than the team. Toil that scales per machine, per customer, or even per feature can quickly overwhelm a small SRE team. When SREs run into this type of toil, they have a few options:

- Eliminate or minimize the toil via automation or changes to system design, processes, or new features. This may require first analyzing what is driving the toil and fixing or removing those factors. As an example, if the toil scales per user, the system may need a mechanism to slow user onboarding, like an invitation-based approach.
- Stop doing the toil completely if it's not necessary. Toil is sometimes added because it has some imagined benefit, such as increased safety or tighter control. However, if evidence shows the benefit is not there, the toil is ineffective and should be stopped.
- Identify a more suitable team that can scale effectively with the toil. This might be the development team, the support team, professional services, or someone else. The key is that if the toil is *necessary* and SRE can't keep up with it, there needs to be another owner.

Moving the work might seem like a false solution, but it's sometimes necessary. Companies hire SREs to be reliability experts and engineers. If toil grows to the point that it starves out the engineering work, it's time to come up with another plan. This plan might be finding a team that can grow to keep up with the work, or finding one that can do the engineering work to eliminate the toil.

Toiling to limit production access

Occasionally, SRE teams will take on toil because the work requires using a tool or permissions that are too dangerous to open up to a broader set of people. However, overpowered tools and broad permissions are dangerous no matter who wields them. Unfortunately, teams have many incentives to create powerful tools that can do too much. These tools are then considered "unsafe" for non-SREs. This scenario is an antipattern and should be avoided.

A better approach is to create sets of tools, each with only one responsibility and limited access. This approach reduces the security and reliability risk of operations. In particular, toil involving customer data or root access should be avoided wherever possible; even if most operators work in good faith, open access to these presents a significant security and reliability risk. Every operation should follow the principle of least privilege.

When tooling and the underlying services are designed well, with clear responsibilities and limited access permissions, the work also becomes easier to automate.

Human Factors in Automation

The automation you create is part of a sociotechnical system that includes people, software, hardware, and processes. When a process is only partially automated, the automation and the operators form a team. Like in any team, there is a division of labor: the automation performs the parts best suited for computers, and the humans do the work that requires human judgment. For this setup to work effectively, human factors such as skills, limitations, biases, and behaviors must be considered.

Since humans are part of partially or incrementally automated systems, it is important to take a human-centered design approach when automating, understanding the capabilities and limitations of the operators to improve the user experience.

User Interface

Usability is a reliability concern in partially automated systems. When outages are attributed to "operator error," it is almost always the case that the tools or automation had poorly defined user interfaces. Common pitfalls are bad default values, cryptic identifiers, ambiguous flag names, and confusing error messages. Taking the time to make sure that the automation is easy to understand and operate is essential for reliability. For critical systems, automation should go through some form of user experience testing.

Observability

Automation, like any software, requires some level of observability. The operators should have the information they need to understand the status of the work, both at a high level and a low level, in case the automation breaks and troubleshooting is needed. Attention to good error messaging, metrics, and logging is key.

Speed of Operations

One significant limitation of humans is that they work much, much more slowly than computers. Thus, a key consideration in human-centered design is whether the speed of the automation will overload the human operators. For example, a common pattern is to have automation offload difficult decisions to a manual queue worked by a human operator. If something changes to cause a significant increase in the rate of decisions routed to the queue, the operators may not be able to keep up.

Human slowness can also hide race conditions in complex processes. When automation speeds up one part of the process, it often reveals challenges in another part. Processes and systems may need to be redesigned to work safely again.

Google encountered this challenge when it took an iterative approach in automating the process by which new cloud regions are brought up. As more and more parts of the process were automated, automation that had worked previously started failing due to hidden assumptions about the order and timing of the steps in the automation. As a result, the process had to be redesigned.

Team Implications

When building automation, SREs need to consider the team's skill levels, experience, and other relevant factors. Automation that introduces a new technology—whether a new data serialization format for configuration or a new authorization system—requires every team member to learn it. In general, automation that requires extensive training and onboarding is ill-suited to a team that is temporary or has high turnover.

Just as automation may force processes and systems to change, it can lead to changes in the makeup of your team and how it interacts. As mentioned in the sidebar "The Ironies of Automation", automation creates new work in the form of monitoring and repairing it, so the team must include people with those skills. As automation makes systems more complex, teams may also find that they need clear escalation paths so less experienced engineers can easily loop in those with more expertise.

Conclusion

In contrast to the first edition, where the chapters on toil and automation focused more on defining the central concepts and guiding philosophies, here we have tried to provide a set of pragmatic considerations to apply while examining your own specific situation. We introduced the nuances that emerge once you incorporate a cost-benefit analysis, including second-order effects such as deskilling and introducing new classes of work. The hallmark of an effective SRE team focused on delivering business value is that it knows *what* tools to reach for *and when*, to best support the business. Thriving in ambiguity is a key cultural value at Google, and even seemingly simple questions such as "Should we automate this?" can lead you down surprising paths.

1 Lisanne Bainbridge, "Ironies of Automation", *Automatica* 19, no. 6 (1983): 775–79.

2 There is a mathematical trade-off between false positives and true positives whenever a static threshold is established (which is typical in automation). Therefore, in the face of ambiguous signals, there is a mathematically required increase in manual intervention and system tuning (and thus maintenance). Maintenance costs are rarely properly accounted for in cost-benefit analysis, so any factor that increases maintenance costs will almost always increase hidden costs that were already not properly accounted for. For more information, see Sachidanandam Sakthivel, "A Critical Review of Cost-Benefit Analysis for Business Software Investments", *American Journal of Business* 38, no. 4 (2023): 229–47.

3 In comparison to GFE (2005), GSLB (2008), and Maglev (2009), discussed in Chapter 11 of *The Site Reliability Workbook*, which are old enough to drive a car and vote in some jurisdictions.

4 For more on multi-single-tenant architecture, see Jennifer Mace, "Multi-Single-Tenant Architectures in Cloud", O'Reilly Content, January 7, 2020.

5 Not to be confused with Kubernetes pods. Google is a big place, and naming collisions are unfortunately common. Sorry!

6 This feat is difficult to accomplish.

7 A notable example is that the size of our inputs broke our Mixed Integer Programming capacity solver.

8 For example, Autocap is a tool that is supposed to do automated capacity management, and to this day we have not managed to make it successfully work for us in this context.

[9] For an in-depth discussion of toil, see Chapter 5 in the first edition of this book and Chapter 6 in *The Site Reliability Workbook*.

Previous chapter
< IV. Infrastructure Design

Next chapter
14. Testing >